**SIMATS ENGINEERING**

**ASSESSMENT TOOL 3**

**COURSE NAME:** DESIGN AND ANALYSIS OF ALGORITHMS
**COURSE CODE: CSA06**

---

**COURSE OUTCOME COVERED**

**CO2:** Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)

Assessment Tool Weightage: 10%

---

**QUESTIONS: 10** **Total Marks:50**

**Annexure A**

**DEBUGGING & OPTIMISATION TASK**

---

***Code of Conduct:***

*I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
*I understand that any violation of academic integrity rules will result in disciplinary action.*

***Signature of the student:***

The given implementation has two logical errors that cause incorrect results.

## Original Code

```
def linear_search(arr, key):
```

```
    for i in range(len(arr)):
        if arr[i] == key:
            return -1   # ERROR
    return i            # ERROR
```

## Errors and Their Causes

### Error 1: Incorrect return value when the element is found

return -1

- **Cause:** `-1` is typically used to indicate that the element was **not found**.
- **Correct behavior:** Return the index `i` where the key is found.

### Error 2: Incorrect return value after the loop

return i

- **Cause:** If the element is not found, `i` will contain the last index of the array, giving the false impression that the element exists there.
- **Additional issue:** If the array is empty, `i` is never assigned, causing an `UnboundLocalError`.
- **Correct behavior:** Return `-1` after the loop completes without finding the element.

---

# Corrected Version

```
def linear_search(arr, key):
    """
    Searches for 'key' in the list 'arr'.

    Parameters:
        arr (list): List of elements.
        key: Element to search for.

    Returns:
        int: Index of the key if found, otherwise -1.
    """
    for i in range(len(arr)):
        if arr[i] == key:
            return i      # Return the index where the key is found
```

```
    return -1           # Key not found
```

---

# More Readable and Pythonic Version

Using `enumerate()` makes the code cleaner because it provides both the index and the element directly.

```
def linear_search(arr, key):
    """
    Perform a linear search for 'key' in 'arr'.

    Returns:
        int: Index of the key if found, otherwise -1.
    """
    for index, value in enumerate(arr):
        if value == key:
            return index

    return -1
```

### Advantages

- More readable.
- Avoids repeated indexing (`arr[i]`).
- Follows Python best practices.

---

# Example

```
numbers = [12, 25, 8, 41, 19]

print(linear_search(numbers, 41))  # Output: 3
print(linear_search(numbers, 15))  # Output: -1
```

#### Output

```
3
-1
```

---

# Time Complexity Analysis

Let **n** be the number of elements in the array.

- **Best Case: O(1)**
  - The key is found at the first element.
- **Average Case: O(n)**
  - On average, about half the elements are checked.
- **Worst Case: O(n)**
  - The key is at the last position or not present, so every element is examined.

### Space Complexity

- **O(1)** (constant extra space)
  - The algorithm uses only a few variables regardless of input size.

---

## Summary

| Issue | Original Code | Correct Code |
|---|---|---|
| Return when key is found | `return -1` | `return i` (or `return index`) |
| Return when key is not found | `return i` | `return -1` |
| Empty list handling | Can raise `UnboundLocalError` | Safely returns `-1` |
| Time Complexity | O(n) | O(n) |
| Space Complexity | O(1) | O(1) |

The corrected implementation properly returns the index when the key is found, returns `-1` when it is absent, handles empty lists safely, and maintains the optimal **O(n)** time complexity for linear search.